Lecture 02: Tabular RL

Petr Kuderov

Researcher at MIPT & AIRI

February 04, 2026

Tabular Reinforcement Learning

- finite MDPs
- tabular state and action spaces
- episodic setting
- lecture + interleaved practice … again
- data: interaction trajectories
- goal: *optimal policy via value estimates* (value-based RL family)

Prologue

- reversed order vs standard RL path
- start from “final” algorithms
- then derive where they come from

- first: what code does
- later: where targets come from
- idea: *define target, reduce error*

Two Core Algorithms: SARSA and Q-learning

- both learn action-value function Q(s, a)
  - table of size $|S| \times |A|$
- both update online from interaction
- SARSA: learn Q^{π}
- Q-learning: learn Q^{π^*}

SARSA & Q-learning (short)

SARSA

Repeat (for each episode):
  Init s
  Select $a \sim \pi(s) = \varepsilon$-greedy on Q
  Repeat (for each step of episode):
    Take action a, observe r, s'
    Select $a' \sim \pi(s') = \varepsilon$-greedy on Q
    $Q(s,a)$ += $\alpha[r + \gamma Q(s',a') - Q(s,a')]$
    $s \leftarrow s'$, $a \leftarrow a'$
    ... until s is terminal

Q-learning

Init $Q(s,a)$ arbitrary, $Q(\text{terminal},\cdot) = 0$
Repeat (for each episode):
  Init s
  Repeat (for each step of episode):
    Select $a \sim \pi(s) = \varepsilon$-greedy on Q
    Take action a, observe r, s'
    $Q()$ += $\alpha[R + \gamma \max Q(s',\cdot) - Q(s,a)]$
    $s \leftarrow s'$
    ... until s is terminal

- similar structure
- key difference: target

General scheme

Initialize $Q(s, a)$ for all $s \in S$, $a \in A(s)$ arbitrarily, $Q(\text{terminal}, \cdot) = 0$

Repeat (for each episode):

  Initialize S

  Repeat (for each step of episode):

    Choose A from S using policy derived from Q (e.g. ε-greedy)

    Take action A, observe R, S'

    $Q(S, A) \leftarrow Q(S, A) + \alpha[G(S', A') - Q(S, A)]$

    $S \leftarrow S'$

    until S is terminal

- Update Q: regular delta-rule learning the target $G(s, a)$

What Is Being Learned?

- Q(s, a): expected long-term return
- policy implicit or explicit
- learning via error correction
- target minus current estimate
- value = expected return estimate
- learning signal: $\delta = \text{target} - Q$
- control: make policy greedy wrt Q

Returns

Return G_t:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots$$

- sum of future rewards
- random variable
- high variance
- delayed signal
- depends on future randomness
- horizon grows variance
- discount: $\gamma \in [0, 1)$

State value:

$$V^\pi(s) = E_\pi[G_t \mid s_t = s] = E_\pi[G(s)]$$

Action value:

$$Q^\pi(s, a) = E_\pi[G_t \mid s_t = s, a_t = a]$$
$$= E_\pi[G(s, a)]$$

- policy: $\pi(a|s)$
- expectation over trajectories
- objective: maximize $E[G_0]$

How to learn V/Q?
aka *evaluate policy*

Policy evaluation option 1: Monte Carlo Evaluation

- **idea**: estimate expected returns by sampling
- for a fixed policy π
- sample episodes
- estimate returns
- update by averaging
- learn from complete episodes
- first-visit vs every-visit
- incremental averaging view

MC update

$$Q(s,a) \leftarrow Q(s,a) + \alpha_t(G_t - Q(s,a))$$

- target = sampled return
- needs episode termination
- episode-level updates
- unbiased
- high variance

Recursive Form of Return

Return G_t:

$$
\begin{aligned}
G_t &= r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots \\
&= r_t + \gamma \left[r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} \right] \\
&= r_t + \gamma G_{t+1}
\end{aligned}
$$

- bootstrapping idea
- local target
- leads to Bellman equations
- one-step lookahead
- can be more than one -step lookahead, i.e. n-step
- bootstrap target uses estimate
- bridge to TD targets

Bellman Expectation Equation

For a fixed policy π:

$$V_\pi(s) = E_\pi[r_{t+1} + \gamma V_\pi(s_{t+1}) \mid s_t = s]$$

$$Q_\pi(s,a) = E[r_{t+1} + \gamma E_\pi[Q_\pi(s_{t+1}, a')]]$$

- maps value function to value function
- contraction
- fixed point: $Q_\pi = T^\pi Q_\pi$
- not an algorithm
- T^π contraction with factor γ
- repeated application converges
- DP uses full expectation

Policy evaluation option 2: Temporal-Difference Learning

$$\text{TD\_target} = r + \gamma Q(s', a')$$

$$\text{TD\_error} = \delta = \text{TD\_target} - Q(s, a)$$

- sample Bellman target
- update every step
- lower variance than MC
- biased but consistent
- bootstrap: use $Q(s', a')$
- online updates per step
- lower variance targets

Practice 1: Q-learning, SARSA, MC-control

From Evaluation to Two-Stage Control scheme

- goal: optimal policy
- improve policy using value estimates
- alternate evaluation and improvement
- evaluation: estimate Q_π
- improvement: greedy wrt Q
- generalized policy iteration idea

- stage 1: policy evaluation
- stage 2: policy improvement
- repeat until stable
- evaluate approximately
- improve greedily
- interleave in practice

Stage 1: Fix Policy

- Q $\rightarrow$ greedy policy $\rightarrow$ epsilon-greedy
- learn Q_π
- need exploration
- behavior vs target viewpoint
- target: greedy, behavior: ε-greedy
- learn on-policy Q^π

Stage 2: Policy Improvement

- $\pi(s) = \arg\max_a Q(s, a)$
- greedy improvement
- local decision
- increases value
- improvement operator
- makes policy sharper

Convergence

- tabular case
- sufficient exploration
- decreasing step sizes
- nuances later
- sufficient exploration: visit all pairs
- step sizes: $\sum \alpha_t = \infty, \sum \alpha_t^2 < \infty$

Collapsing Stage 1

- evaluation and improvement now interleaved
- explic policy not explicit
- greedy action selection inside loop
- policy improvement inside data collection
- evaluation never fully converged
- GPI loop: policy/value co-adapt

Back to SARSA

$$Q(s,a) \leftarrow Q(s,a) + \alpha(r + \gamma Q(s',a') - Q(s,a))$$

- on-policy
- target follows behavior
- on-policy TD control
- learns value of behavior policy
- sensitive to exploration in risky MDPs

SARSA scheme

SARSA

Initialize $Q(s,a)$ for all $s \in S$, $a \in A(s)$ arbitrarily, $Q(\text{terminal},\cdot) = 0$

Repeat (for each episode):

  Initialize S

  $a \sim \pi_\varepsilon(s)$: Choose a from s using policy derived from Q (e.g. ε-greedy)

  Repeat (for each step of episode):

    s', r' = env.step(a): Take action a, observe r, s'

    $a' \sim \pi_\varepsilon(s')$: Choose a' from s' using policy derived from Q (e.g. ε-greedy)

    $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)]$

    $s \leftarrow s'$, $a \leftarrow a'$

    until s is terminal

Expected SARSA

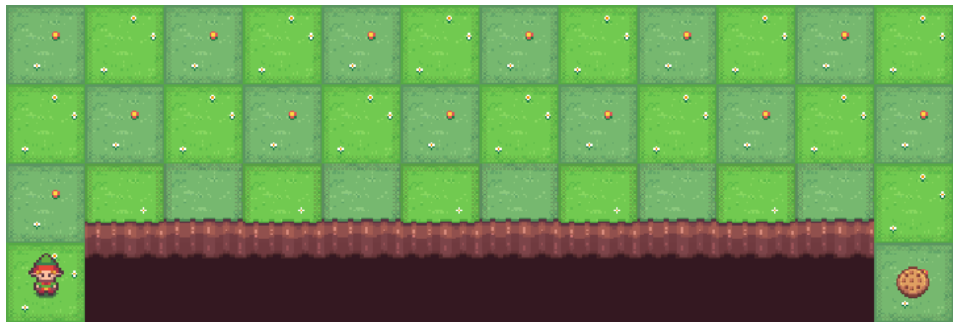
Replace TD target with:

$$\begin{aligned}\text{TD\_target} &= r + \gamma \mathbb{E}_{a' \sim \pi_\varepsilon}[Q(s', a')] \\ &= r + \gamma \sum_{a'} \pi_\varepsilon(a'|s') Q(s', a')\end{aligned}$$

- expectation instead of sample
- **lower variance**
- more computation
- bridges SARSA and Q-learning

Cliff Walking Example

- goal: reach RB cell from LB start
- dangerous states: high penalty + termination
- optimal behavior: shortest path
  - it's risky under exploration
- resulting SARSA policy depends on ε
  - higher $\varepsilon \Rightarrow$ "safer" path



- During training exploration is turned on: π_ε^Q. When we turn it off to π^Q, is it optimal?
- $Q^{\text{what pi???}}(s, a)$
- Does SARSA differentiate behavior and target policies?

How could we “fix” that?

Option 1:

Set $\varepsilon \to 0$ decay during training. In limit, it converges to optimal greedy policy.

Option 2:

Instead of learning Q^{π_ε} for behavior policy, learn it for the target (=greedy) policy: Q^{π}.

How can we learn Q for the target greedy policy?

$$\text{exploration behavior expectation: TD\_target} = r + \gamma \mathbb{E}_{a' \sim \pi_\varepsilon} Q(s', a')$$

$$\Downarrow$$

$$\text{greedy behavior expectation: TD\_target} = r + \gamma \mathbb{E}_{a' \sim \pi} Q(s', a')$$

$$= r + \gamma Q\left(s', \arg\max_{a'} Q(s', a')\right) = r + \gamma \max_{a'} Q(s', a')$$

Bellman Optimality Equation

$$Q^*(s,a) = E[r + \gamma \max_{a'} Q^*(s',a')]$$

- optimal control objective
- optimality operator T^*
- combines evaluation + improvement
- fixed point: $Q^* = T^*Q^*$

- on-policy:
  - target = behavior
  - Bellman expectation operator
- off-policy:
  - target != behavior
  - Bellman optimality operator

Q-learning

Q-learning

Initialize $Q(s,a)$ for all $s \in S$, $a \in A(s)$ arbitrarily, $Q(\text{terminal},\cdot) = 0$

Repeat (for each episode):

  Initialize S

  Repeat (for each step of episode):

  $a \sim \pi_\varepsilon(s)$: Choose a from s using policy derived from Q (e.g. ε-greedy)

    s', r' = env.step(a): Take action a, observe r, s'

    $Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$

    $s \leftarrow s'$

    until s is terminal

SARSA vs Q-learning

On-policy vs Off-policy

Q-function for

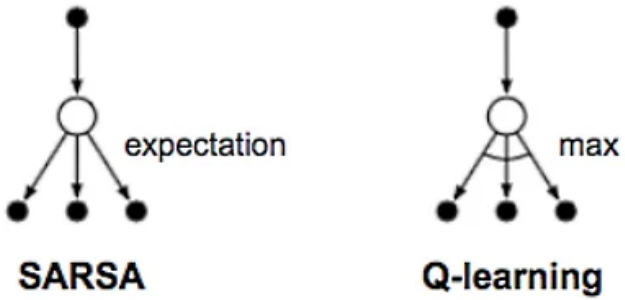
- SARSA: behavior policy
- Q-learning: greedy policy

Update

- SARSA: sample from π
- Expected SARSA: average over π
- Q-learning: maximize over actions

Different optimization force

- Bellman expectation operator doesn't optimize Q-induced policy
- SARSA: soft-greedy behavior
- Q-learning: Bellman optimality operator



Off-policy

Off-policy

- differentiates behavior and target policies
- enables data reuse (replay buffer)
- tabular case: strong convergence + optimality guarantees
- can be unstable with approximation (deadly triad)
  - data distribution is mismatched
  - projection norm is incorrect
  - breaks operator

Offline RL (Spoiler)

- learn from fixed data
- possible with off-policy methods
- powerful, but many pitfalls
- fixed dataset, no exploration
- distribution shift issues
- extrapolation error

Wrap-up: Control Methods

- MC control vs SARSA vs Q-learning
- same structure, different targets
- all are GPI variants
- differ by target definition
- compare: bias/variance, risk, data reuse

Bellman Operators

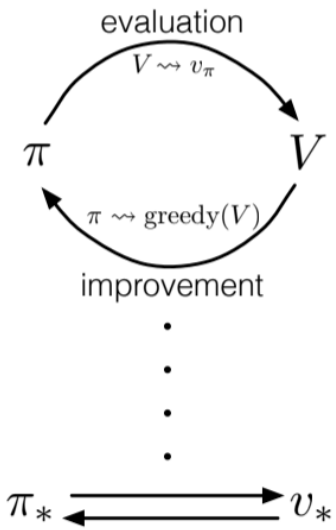
- expectation: $(T^{\pi}Q)(s,a) = E[r + \gamma Q(s', a')]$
- optimality: $(T^{*}Q)(s,a) = E[r + \gamma \max_{a'} Q(s', a')]$
- expectation operator
- optimality operator
- contraction mappings
- fixed points
- sampling gives TD targets

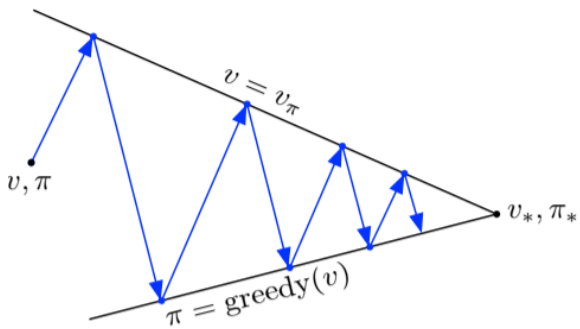
Policy Iteration and Value Iteration

- simpler setting: planning with known model
- evaluation + improvement
- PI: T^π to convergence + greedy and again
- VI: repeated T^* updates

In practice

- small tabular MDP
- full Bellman backups
- compare convergence
- synchronous vs asynchronous backups
- stop criteria: max change < tol
- compare iterations vs cost





Takeaway

- define target
- move value toward target
- control via improvement pressure
- differences are structural
- choose target: MC vs TD vs optimality
- choose coupling: on-policy vs off-policy
- one template: target, error, update